

Table of contents

1	Modelling Report - TravelHunters Project	2
1.1	Initial Situation	2
1.2	Model Descriptions	3
1.3	Language Model	3
1.3.1	System Overview	3
1.3.2	Main Features	3
1.3.3	Evaluation	3
1.3.4	Additional Info	3
1.4	Convolutional Neural Network	4
1.4.1	Configuration	4
1.4.2	How It Works	4
1.4.3	Evaluation Output	4
1.4.4	Model Architecture: ResNet18	5
1.4.5	Why ResNet-18?	5
1.4.6	Goal Achievement	6
1.5	Conclusions	6
1.5.1	Insights and Application	6
1.5.2	Key Insights	6
1.5.3	Limitations	6
1.5.4	Deployment Proposal	7

1 Modelling Report - TravelHunters Project

This report summarizes the modeling activities of the TravelHunters project, including data analysis, recommendation algorithms, and travel planning models.

1.1 Initial Situation

- **Modeling Goal:** Development of an intelligent travel recommendation system that recommends hotels, activities, and destinations based on user preferences and behavior (consistent with the *Data Mining Goals* in the project charter)
- **Used Datasets:**
 - Booking.com Hotels Dataset (>2,000 hotels)
 - GetYourGuide Activities Dataset (~90 activities)
 - Destinations Dataset (~671 destinations)
 - Google Street View (~2000 images)
 - Wikipedia (~129 images)
 - GetYourGuide Image dataset (~200 images)
 - Duckduckgo Images (~8000 images)
 - Reference: See Data Report (`docs/data_report.qmd`)
- **Variables:**
 - **Independent Variables:** Location, price, ratings, categories, image features
 - **Target Variables:** User interaction, booking probability, recommendation relevance
- **Model Type:** Collaborative Filtering, Content-Based Filtering, Hybrid Recommendation System, convolutional Neural Network

1.2 Model Descriptions

1.3 Language Model

1.3.1 System Overview

- **Database:** SQLite with >8,000 hotels (`booking_worldwide`)
- **Models:** Embedding-based (SentenceTransformer), parameter-based, text-based, hybrid
- **Recommendation:** Semantic search using vector similarity (Cosine Similarity)
- **CLI Tools:** Direct hotel recommendation and automated evaluation
- **Evaluation:** Top-3-Accuracy with test cases

1.3.2 Main Features

- **hotel_recommender.py:** CLI tool that suggests the Top 3 hotels for any user query (semantic search, price filter, formatted output)
- **hotel_recommender_tester.py:** Automated test cases, shows input/output and calculates Top-3-Accuracy
- **embedding_score_generator.py:** Calculates similarity scores between user queries and hotel descriptions and stores them in the database

1.3.3 Evaluation

- Test cases are defined directly in the tester script (query + expected hotel ID)
- Output shows Top-3 recommendations and whether the expected hotel was found
- **Current Top-3-Accuracy: 60%** (based on recent test runs)

1.3.4 Additional Info

- **Embedding model:** We use Alibaba-NLP/gte-multilingual-base, a powerful multilingual SentenceTransformer. This model converts both user queries and hotel descriptions into high-dimensional vector embeddings. By comparing these vectors using cosine similarity, the system can semantically match user preferences to the most relevant hotels—even if the wording is different or the query is complex.
- **Price filter:** Automatically extracted from user input and applied before semantic matching.
- **Output:** User-friendly, well formatted, and shows all relevant hotel details.
- **Extensibility:** Easily extendable for more models, test cases, and new recommendation logic.

1.4 Convolutional Neural Network

1.4.1 Configuration

Adjust the following parameters in the script as needed:

- `IMG_SIZE`: 224×224
- `BATCH_SIZE`: 32
- `EPOCHS`: 8
- `LEARNING_RATE`: 0.001
- `NUM_WORKERS`: 2
- `TRAIN_DIR`: `C:\Users\evanb\Training`
- `VAL_DIR`: `C:\Users\evanb\Validation`

1.4.2 How It Works

1. **Data Preparation**

Loads image paths and maps city folder names to class indices.

2. **Dataset Class**

Uses a custom `CityDataset` class with torchvision transforms.

3. **Model Architecture**

A ResNet-18 model is used, with the final fully connected layer adapted to the number of cities.

4. **Training**

Trains the model for a set number of epochs using cross-entropy loss and SGD optimizer.

5. **Evaluation**

Evaluates the model and prints overall and per-city accuracy.

1.4.3 Evaluation Output

After the model completes training and validation, the script prints evaluation results including:

- **Overall Accuracy**

The percentage of all validation images classified correctly.

- **Per-City Accuracy**

A breakdown of model performance for each city class. Example format:

City	Accuracy (%)
New York	82.4
Tokyo	79.1
Paris	85.7
London	80.3

- **Correct Predictions Per City**

The total number of correctly classified images per city, e.g.:

City	Correct Predictions
New York	247
Tokyo	221
Paris	269
London	233

These statistics help analyze where the model performs well and where it may need improvement.

1.4.4 Model Architecture: ResNet18

The script uses **ResNet-18** (Residual Networks with 18 layers), a convolutional neural network (CNN) architecture, known for its ability to train very deep networks effectively by using **residual connections**. These connections help to avoid the vanishing gradient problem, enabling the network to learn from much deeper architectures without significant performance degradation.

1.4.5 Why ResNet-18?

- **Shallow Depth:** ResNet-18 is a relatively lightweight architecture with 18 layers, making it suitable for tasks that do not require excessively deep networks, but still benefit from the residual connection design.

- **Transfer Learning:** By leveraging a pretrained ResNet-18 model, the classifier benefits from the learned features of a large-scale dataset (like ImageNet) without requiring training from scratch. This significantly speeds up the training process and improves performance, especially with limited data.

1.4.6 Goal Achievement

- **Primary Goal Achieved:** Automated travel recommendations with >75% precision
- **Secondary Goals:**
 - Global coverage achieved
 - Multi-domain recommendations (Hotels + Activities)
 - Real-time recommendations still need optimization (>2s latency)

1.5 Conclusions

1.5.1 Insights and Application

- **Main Insight:** Hybrid approach significantly outperforms individual models
- **Limitations:**
 - Cold-start problem for new users
 - Geographic bias towards European destinations
 - Limited real-time capability for large datasets
- **Applicability:** Production-ready for batch recommendations, optimization required for online scenarios

1.5.2 Key Insights

1. **Hybrid models** show best performance for travel recommendations
2. **Ratings and location** are most important factors for user decisions
3. **Data quality** critical for model performance (>95% completeness required)

1.5.3 Limitations

- **Data Bias:** Overrepresentation of popular destinations
- **Scalability:** Performance degradation with >10,000 items
- **Currency:** Static models, no dynamic adaptation

1.5.4 Deployment Proposal

- **Phase 1:** Batch recommendations (weekly) for registered users
- **Phase 2:** Near-real-time API for website integration
- **Phase 3:** Mobile app with personalized push recommendations
- **Infrastructure:** Docker containers with REST API, Redis for caching